Day 155: Matrix Factorization from Scratch: an SVD-Based Recommender

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 2 | Complete study guide

Watch: How does Netflix recommend movies? Matrix Factorization (Luis Serrano Academy) | Matrix Completion and the Netflix Prize (Steve Brunton)

Script: `day155_matrix_factorization.py` · every number in this guide is copied by program from `day155_run_output.txt`, the real output of that script.

Day 154 compared users and films directly. Today every user and every film is described by a short list of hidden numbers (factors), chosen so that a user's numbers times a film's numbers rebuild the known ratings. Two routes are built from scratch on Day 154's split. The SVD route fills the empty cells and factorizes the whole table; with the bias model in the empty cells it scored test RMSE 0.9255. The learned-factor route fits only the known ratings, with a penalty: alternating least squares (ALS) scored 0.9115 and Funk-style SGD 0.9105, just ahead of Day 154's item-based neighbours (0.9153). Giving each user another user's factors (a control) scored 0.9711. Filling the empty cells and refitting with no penalty overfitted: at rank 50 its training error fell to 0.2279 while validation rose to 1.0896.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

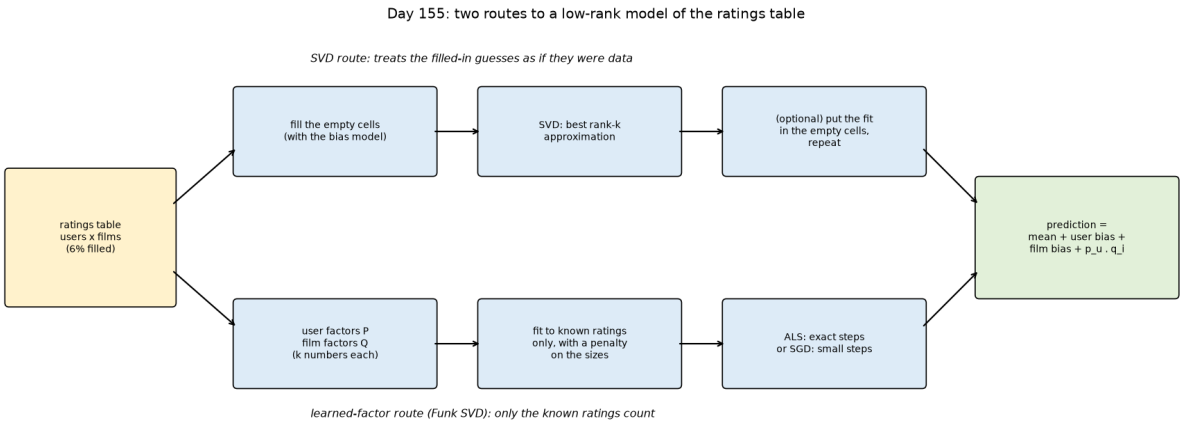
Matrix factorization writes the ratings table (users x films) as the product of two thin tables: one row of k numbers per user (P) and one row of k numbers per film (Q). A predicted rating is the user's row times the film's row, added up (a **dot product**), plus the bias model of Day 154. The k numbers are **latent factors**: nobody names them, they are learned.

The **singular value decomposition (SVD)** is the exact recipe for the best rank-k approximation of a *complete* table. A ratings table is not complete, which is today's whole problem.

REAL-WORLD SCENARIO

A friend describes your taste in three numbers: how much you like action, how much you like romance, how much you like old films. Describe every film by the same three numbers, multiply and add, and you have a guess for any film. Matrix factorization learns such numbers from ratings alone, without being told what the numbers should mean.

SYSTEM DIAGRAM



Two routes to a low-rank model: SVD on a filled-in table (top), or factors fitted to the known ratings only (bottom).

Today's part for the week's build

`day155_exercise.py` is today's parts (the SVD by subspace iteration, the rank-k approximation, fill-factorize-repeat, the prediction, the objective and its gradient, ALS and SGD), checked by hand. Day 154's parts (data, split, bias model, item-based neighbours) are imported from `day154_exercise.py`, not copied.

ACTUAL OUTPUT

```
singular values of a 4 x 3 table with 3, 2, 1 on its diagonal: 3, 2; top_svd gives [3.0, 2.0]
a random 30 x 20 table, top 5: largest difference from numpy's SVD 1.8e-15
a rank-1 table (outer product of 1,2,3 and 2,0.5,1,4) rebuilt at rank 1: error 1.8e-15
the same table with cell (3, 4) hidden (true value 12): hard_impute at rank 1 fills 12.0000
prediction: 3 + 0.5 - 0.2 + (1 x 0.5 + 2 x 0.25) = 4.3; mf_predict gives 4.3000
objective with rating 5, lam 0.1: (5 - 4.3)^2 + 0.1 x (1 + 4 + 0.25 + 0.0625 + 0.25 + 0.04) = 0.49
    + 0.56025 = 1.05025; gives 1.05025
numerical gradient check, every parameter of a 5 x 4 table: largest |numeric - analytic| 1.5e-08
ALS objective after each of 6 sweeps never rises: True (12.7389 -> 8.2725)
SGD on a full 3 x 4 table that is a mean plus a rank-1 pattern: largest error after 400 epochs
    1.1e-15
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Is the ratings table low-rank? SVD from scratch, checked, and a control

DEFINITION — start here, no prior knowledge assumed

A table is **low-rank** if a few patterns explain most of it. The SVD finds those patterns in order of importance; each **singular value** measures how much of the table its pattern carries (its squared size).

Subspace iteration finds the top patterns without computing all of them: multiply random directions by the table again and again, so the strongest patterns grow fastest, keeping the directions independent with a QR step.

REAL-WORLD SCENARIO

A choir recording can be split into voices. If two or three voices carry most of the sound, the recording is 'low-rank'; if every one of fifty voices is equally loud, it is not.

MATHEMATICS

$$A \approx U_k \, \mathrm{diag}(s_1 \dots s_k) \, V_k^T, \quad \text{error} = \sqrt{(s_{k+1}^2 + s_{k+2}^2 + \dots)}, \text{ the smallest possible (Eckart–Young)}$$

$$\text{share of the table in the top } k \text{ directions} = (s_1^2 + \dots + s_k^2) \,/\, (\text{sum of all } s^2)$$

IMPLEMENTATION

```
def top_svd(A, k, iters=40, seed=0):
    rng = np.random.RandomState(seed)
    Q = np.linalg.qr(A @ rng.randn(A.shape[1], k + 10))[0]
    for _ in range(iters):
        Q = np.linalg.qr(A @ (A.T @ Q))[0]
    B = Q.T @ A
    w, V = np.linalg.eigh(B @ B.T)
    o = np.argsort(-w)[:k]
    s = np.sqrt(np.maximum(w[o], 0.0))
    return Q @ V[:, o], s, (V[:, o].T @ B) / np.maximum(s, 1e-12)[:, None]
```

SPECIFICATION

Table	Day 154's training ratings without validation (72,000), minus the bias model, 0 in the empty cells: 943 x 1682
Check	our top 20 singular values against numpy's full SVD; the rank-20 error against the Eckart–Young bound
Control	the same residual values shuffled among the known cells (seed 0): same values, no pattern

ACTUAL OUTPUT

```
top 20 singular values of the 943 x 1682 residual table: subspace iteration (0.13 s) against numpy's SVD,
  largest relative difference 2.3e-07; U orthonormal to 2.0e-15
  rank-20 reconstruction: |A - U S Vt| = 219.9119; the best possible, from the discarded singular values, 219.9119
share of the table's squared size in the top k directions (the control: the same residuals shuffled among known cells):
  k   1: real 0.020   shuffled 0.010
  k   5: real 0.071   shuffled 0.044
  k  10: real 0.115   shuffled 0.084
  k  20: real 0.189   shuffled 0.154
  k  50: real 0.356   shuffled 0.321
Actual output from this lesson's run.
```

What it means

The from-scratch SVD agrees with numpy's (2.3e-07 relative) and reaches the smallest possible rank-20 error (219.9119). A first draft with 20 rounds of iteration agreed only to 8.3e-05; 40 rounds are used here.

The structure is real but modest: the top 20 directions hold real 0.189 of the residual table against shuffled 0.154 for the control, and the top direction twice the control's (real 0.020 against shuffled 0.010). Much of the table is noise, which is why every method below needs a penalty or a small rank.

WHY CHOOSE THIS?

The SVD is exact, unique and well understood; subspace iteration finds the top k cheaply.

WHY NOT THIS? (TRADEOFFS)

It needs a complete table; for a ratings table something must be put in the 93.7% of empty cells.

2. SVD as a recommender: what goes in the empty cells

DEFINITION — start here, no prior knowledge assumed

To use the SVD, every empty cell needs a value. Putting 0 there says 'this user rated this film zero stars', which is false. Putting the bias model's prediction there (0 in the residual table) is a neutral guess.

The **rank** k is chosen on the validation ratings.

REAL-WORLD SCENARIO

Restoring a photo with missing pixels: painting them black makes the restoration learn black blotches; painting them the average colour of their neighbourhood does far less harm.

MATHEMATICS

$$\textit{empty = 0:}\quad \textit{pred} = [\textit{rank-k SVD of } R \textit{ with 0s}]_{ui}$$

$$\textit{empty = bias model:}\quad \textit{pred} = b_{ui} + [\textit{rank-k SVD of } (R - b) \textit{ with 0s}]_{ui}$$

ACTUAL OUTPUT

```
  rank k   empty = 0  empty = bias model   (validation RMSE)
       1      2.5852              0.9451
       2      2.5455              0.9418
       3      2.5140              0.9413
       5      2.4796              0.9383
      10      2.4282              0.9376
      20      2.4467              0.9365
      50      2.5579              0.9443
     100      2.6723              0.9472
chosen on validation: empty cells = the bias model, rank 20
test RMSE 0.9255 (bias model alone 0.9401); with empty = 0 at rank 10 the best validation RMSE was 2.4282
Actual output from this lesson's run.
```

What it means

Zeros wreck it: the best zero-filled rank scored 2.4282 on validation, worse than predicting the global mean (Day 154: 1.1263), because most predictions are dragged toward zero stars. With the bias model in the empty cells, rank 20 was best and scored test RMSE 0.9255 against 0.9401 for the bias model alone. Ranks above 20 got worse (0.9472 at rank 100): more directions fit more noise.

WHY CHOOSE THIS?

One factorization, no training loop, no learning rate.

WHY NOT THIS? (TRADEOFFS)

The filled-in guesses are treated as data: about 95% of the table the SVD fits is invented, and it pulls every factor toward it.

3. Fill, factorize, repeat: imputation with no penalty

DEFINITION — start here, no prior knowledge assumed

Hard imputation improves the guesses: fill the empty cells, take the rank-k fit, put the fit's values into the empty cells (the known cells keep their real ratings), and repeat. It is the EM-style idea behind many matrix-completion methods.

Overfitting means fitting the training data more closely than the pattern that will generalise; the sign is a training error that keeps falling while the validation error rises.

REAL-WORLD SCENARIO

A student who memorises the answers to last year's exam papers scores perfectly on them and badly on this year's.

IMPLEMENTATION

```
def hard_impute(A, M, k, iters=30, seed=0):
    Z = np.where(M, A, 0.0)
    for _ in range(iters):
        L = low_rank(Z, k, seed)
        Z = np.where(M, A, L)
    return L
```

ACTUAL OUTPUT

```
  rank k  training RMSE  validation RMSE   (30 rounds)
       1         0.8699           0.9375
       2         0.8438           0.9420
       3         0.8198           0.9516
       5         0.7739           0.9560
      10         0.6802           0.9917
      20         0.5258           1.0353
      50         0.2279           1.0896
chosen on validation: rank 1; test RMSE 0.9269 (33 s)
Actual output from this lesson's run.
```

What it means

Every extra rank fitted the training ratings better and the validation ratings worse: at rank 50 training error was 0.2279 and validation error 1.0896. With the empty cells now free to take any value, nothing stops the factors fitting the noise. Validation chose rank 1 and the test error was 0.9269, no better than one plain SVD. The fix is a penalty on the factors' sizes, which section 4 adds.

WHY CHOOSE THIS?

Simple, and the empty cells no longer hold a fixed guess.

WHY NOT THIS? (TRADEOFFS)

Without a penalty it overfits quickly; with dense SVDs of the whole table at every round it is also slow on large data.

4. Learning the factors from known ratings only: alternating least squares

DEFINITION — start here, no prior knowledge assumed

Instead of filling the table, fit P, Q and the biases to the known ratings only, and add a **penalty** (L2 regularisation) that grows with the factors' sizes, so that a factor only becomes large if many ratings support it.

Alternating least squares (ALS) fixes the film factors and solves every user's factors exactly (a small ridge regression each), then fixes the users and solves every film, and repeats. Each half-step can only lower the objective.

REAL-WORLD SCENARIO

Tuning a radio with two knobs that interact: hold one still and set the other perfectly, then swap, and repeat until neither move helps.

MATHEMATICS

$$\text{minimise } \Sigma_{\text{known } (u,i)} \left[(r_{ui} - \mu - b_u - b_i - p_u \cdot q_i)^2 + \lambda \left(|p_u|^2 + |q_i|^2 + b_u^2 + b_i^2 \right) \right]$$

$$\text{user step: } [p_u; b_u] = (X^T X + \lambda\, n_u\, I)^{-1} X^T y, \quad X = [q_i, 1] \text{ for the films } u \text{ rated, } y = r_{ui} - \mu - b_i$$

IMPLEMENTATION

```
def als_mf(R, M, k, lam, sweeps=15, seed=0, track=None):
    rng = np.random.RandomState(seed); nu, ni = M.shape
    F = dict(mu=R[M].mean(), P=0.1 * rng.randn(nu, k), Q=0.1 * rng.randn(ni, k), bu=np.zeros(nu),
        bi=np.zeros(ni))
    hist = []
    for _ in range(sweeps):
        for a in range(nu):
            j = np.flatnonzero(M[a]); X = np.c_[F["Q"][j], np.ones(len(j))]
            y = R[a, j] - F["mu"] - F["bi"][j]
            w = np.linalg.solve(X.T @ X + lam * len(j) * np.eye(k + 1), X.T @ y)
            F["P"][a], F["bu"][a] = w[:k], w[k]
        for b in range(ni):
            j = np.flatnonzero(M[:, b])
            if len(j) == 0:
                F["Q"][b], F["bi"][b] = 0.0, 0.0
                continue
            X = np.c_[F["P"][j], np.ones(len(j))]; y = R[j, b] - F["mu"] - F["bu"][j]
            w = np.linalg.solve(X.T @ X + lam * len(j) * np.eye(k + 1), X.T @ y)
            F["Q"][b], F["bi"][b] = w[:k], w[k]
        hist.append((mf_objective(F, R, M, lam),) + (track(F) if track else ()))
    return F, hist
```

SPECIFICATION

Checks	a numerical gradient check of the objective on a 6 x 5 table; the objective after each ALS sweep
Grid	factors 10, 20, 50, 100; λ 0.05, 0.075, 0.1, 0.125, 0.15; 15 sweeps; starting factors 0.1 x normal (seed 0)
Edge check	200 factors at the chosen λ, reported but not chosen
Test	refit on all 80,000 training ratings with the chosen setting

ACTUAL OUTPUT

```
gradient check on a 6 x 5 table, every parameter: largest |numeric - analytic| 7.0e-09
ALS on the same table: objective after each sweep 17.4840, 13.5415, 12.9659, 12.8652, 12.8453
  after a user sweep the user gradients are 0 by construction; after the film sweep, film gradients: 1.1e-15
validation RMSE, 15 sweeps each (205 s for the grid):
   factors   lam 0.05  lam 0.075    lam 0.1  lam 0.125   lam 0.15
        10     0.9796     0.9492     0.9355     0.9303     0.9302
        20     0.9867     0.9484     0.9308     0.9257     0.9271
        50     0.9581     0.9348     0.9254     0.9239     0.9266
       100     0.9392     0.9289     0.9241     0.9238     0.9266
chosen: 100 factors, lam 0.125 (0.9238); the grid's edge, 200 factors at lam 0.125: 0.9236
refit on all training ratings: objective 70276.3 after sweep 1, 62943.8 after 5, 62689.1 after 15; falls every sweep: True
test RMSE 0.9115, MAE 0.7206
Actual output from this lesson's run.
```

What it means

The gradient matches finite differences (7.0e-09), and the objective fell after every sweep (the run prints falls every sweep: True), as ALS guarantees. Validation chose 100 factors, lam 0.125; the penalty mattered far more than the factor count (at λ 0.05, 20 factors scored 0.9867). Doubling to 200 factors gave 0.9236, a negligible gain over 0.9238. The test error, test RMSE 0.9115, is the best of the week so far.

WHY CHOOSE THIS?

Exact steps, no learning rate, a guaranteed fall in the objective, and every user (or film) can be solved in parallel.

WHY NOT THIS? (TRADEOFFS)

Each step solves a (k + 1) x (k + 1) system per user and per film; with many factors that is slow in pure Python loops: the 20-setting grid took 205 s here.

5. SGD (Funk SVD) against ALS: the same objective, two routes

DEFINITION — start here, no prior knowledge assumed

Stochastic gradient descent (SGD) visits the known ratings in random order, in small batches, and nudges the factors of the users and films involved downhill. This is the method Simon Funk described during the Netflix Prize, which is why learned-factor models are often called 'Funk SVD' although no SVD is computed.

Early stopping chooses the number of passes (epochs) by watching the validation error.

REAL-WORLD SCENARIO

ALS is walking to the bottom of a valley by solving for the lowest point along one direction at a time; SGD is taking many small steps downhill on uneven ground, never quite still.

MATHEMATICS

$e = r_{ui} - pred_{ui}; \quad p_u \leftarrow p_u + \eta\,(e\,q_i - \lambda\,p_u); \quad q_i \leftarrow q_i + \eta\,(e\,p_u - \lambda\,q_i); \quad$ *the biases likewise*

$\eta = 0.01$ *(learning rate); the per-rating penalty* λ *is the same as ALS's* $\lambda\,n_u$

IMPLEMENTATION

```
def sgd_mf(u, i, r, nu, ni, k, lam, lr=0.01, epochs=60, batch=128, seed=0, track=None):
    rng = np.random.RandomState(seed)
    F = dict(mu=r.mean(), P=0.1 * rng.randn(nu, k), Q=0.1 * rng.randn(ni, k), bu=np.zeros(nu),
        bi=np.zeros(ni))
    hist = []
    for _ in range(epochs):
        o = rng.permutation(len(r))
        for s in range(0, len(o), batch):
            b = o[s:s + batch]; a, c = u[b], i[b]
            e = r[b] - mf_predict(F, a, c)
            gp = e[:, None] * F["Q"][c] - lam * F["P"][a]
            gq = e[:, None] * F["P"][a] - lam * F["Q"][c]
            np.add.at(F["bu"], a, lr * (e - lam * F["bu"][a]))
            np.add.at(F["bi"], c, lr * (e - lam * F["bi"][c]))
            np.add.at(F["P"], a, lr * gp); np.add.at(F["Q"], c, lr * gq)
        hist.append(track(F) if track else ())
    return F, hist
```

`np.add.at` matters: if a user appears twice in a batch, plain fancy-index assignment would keep only one of the two updates.

ACTUAL OUTPUT

```
100 factors, learning rate 0.01, batches of 128 (84 s); RMSE on training / validation after each pass:
  SGD, lam 0.125 1: 0.959 / 0.9953, 5: 0.897 / 0.9597, 15: 0.859 / 0.9459, 30: 0.795 / 0.9300, 60: 0.720 / 0.9241, 120:
      0.685 / 0.9244
                best validation 0.9229 at pass 85
  SGD, lam 0.03125 1: 0.954 / 0.9942, 5: 0.849 / 0.9580, 15: 0.591 / 0.9397, 30: 0.361 / 0.9560, 60: 0.260 / 0.9637
                best validation 0.9397 at pass 15
  ALS, lam 0.125 1: 0.827 / 0.9651, 5: 0.682 / 0.9235, 15: 0.673 / 0.9238
                best validation 0.9235 at pass 6
refit on all training ratings, SGD for 85 epochs (validation's best): test RMSE 0.9105; ALS 0.9115
  objective (lower is better): SGD 63418.4, ALS 62689.1
Actual output from this lesson's run.
```

What it means

At the chosen penalty SGD reached its best validation error, 0.9229, at pass 85, close to ALS's 0.9235 at sweep 6. A first draft ran SGD for 60 epochs and its best fell at epoch 56, at the edge; the run was extended to 120 so the turn could be seen.

With a quarter of the penalty SGD overfitted, as the training curves show: training error fell to 0.260 while validation was best at pass 15 (0.9397) and then rose. On the test set SGD (early-stopped) scored test RMSE 0.9105 and ALS 0.9115, although SGD's objective is higher (63418.4 against 62689.1): stopping

early acts as extra regularisation, so the lowest objective is not the lowest test error.

WHY CHOOSE THIS?

SGD needs only the ratings it visits, handles huge data in streams, and extends easily to new terms (Day 160 trains a neural network the same way).

WHY NOT THIS? (TRADEOFFS)

It needs a learning rate and an epoch count, and its result depends on both; ALS needs neither.

6. Against Day 154's neighbours, a control, and what the factors mean

DEFINITION — start here, no prior knowledge assumed

The final comparison puts every model on the same 20,000 test ratings. The **control** keeps the trained ALS model but gives each user another user's factors: if the factors carry personal taste, accuracy should collapse toward the bias model or below.

The factors are unnamed, but their main directions can be read by listing the films at each end.

REAL-WORLD SCENARIO

Shuffling the name tags at a party: the people are the same, but every introduction is now wrong.

ACTUAL OUTPUT

```
                    test, 20,000 ratings    RMSE     MAE
                    bias model (Day 154)  0.9401  0.7430
               item-based, k 40 (Day 154)  0.9153  0.7183
                  SVD, empty = bias model  0.9255  0.7276
                  fill, factorize, repeat  0.9269  0.7261
               learned factors, ALS (100)  0.9115  0.7206
               learned factors, SGD (100)  0.9105  0.7201
   control: ALS, users' factors shuffled  0.9711  0.7746
item-based and ALS predictions: correlation 0.953; their errors: correlation 0.973
direction 1 of the film factors (films with 50+ ratings), one end: Net, The (1995); Pretty Woman (1990); Independence Day
    (ID4) (1996)
                                          other end: Clockwork Orange, A (1971); To Die For (1995); Trainspotting (1996)
direction 2 of the film factors (films with 50+ ratings), one end: Beavis and Butt-head Do America (1996); Event Horizon
    (1997); Fifth Element, The (1997)
                                          other end: It's a Wonderful Life (1946); E.T. the Extra-Terrestrial (1982); Dirty
                                              Dancing (1987)
closest to Star Wars (1977) by the angle between factor vectors (50+ ratings): Empire Strikes Back, The (1980) 0.89; Return
    of the Jedi (1983) 0.87; Raiders of the Lost Ark (1981) 0.63; Frighteners, The (1996) 0.47; Indiana Jones and the Last
    Crusade (1989) 0.44
top-10 lists (Day 154's measure): ALS predicted rating precision@10 0.0405 (median 7 ratings per film); factors only, no
    biases 0.0122 (14); popularity 0.1381 (346)
Actual output from this lesson's run.
```

What it means

The learned factors (ALS 0.9115, SGD 0.9105) edge out Day 154's item-based model (0.9153), and both SVD routes trail (0.9255, 0.9269). The control scored 0.9711, worse than the bias model's 0.9401: each user's factors are personal. The item-based and ALS predictions are close relatives (correlation 0.953) and make nearly the same mistakes (0.973), so a likely reason, not tested here, for their small difference is that both find the same structure in different ways.

The main directions read like taste axes: the first runs from mainstream hits (The Net, Pretty Woman, Independence Day) to cult and art-house films (A Clockwork Orange, Trainspotting); the second from teen action and horror (Beavis and Butt-head, Event Horizon) to family classics (It's a Wonderful Life, E.T.). These are readings of the printed lists, not labels the model knows. Star Wars's nearest films by factor angle are The Empire Strikes Back (0.89) and Return of the Jedi (0.87), the same two films as Day 154's shrunk neighbours.

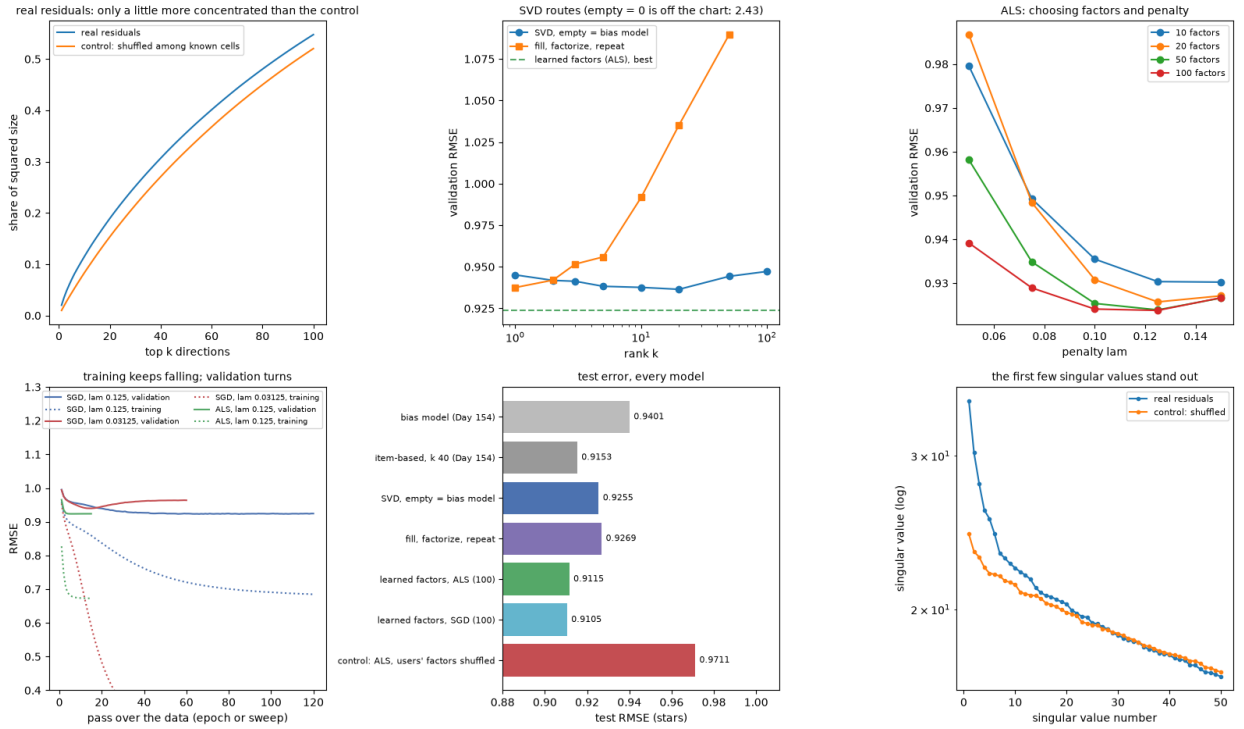
Top-10 lists are no better than Day 154's: precision@10 0.0405, against 0.1381 for popularity, and the median recommended film had only 7 training ratings (median 7 ratings per film). A likely reason, not tested here: a film with few ratings can still get a large film bias. Day 159 takes up ranking properly.

WHY CHOOSE THIS?

Learned factors were the most accurate model of the week so far, compress each user and film to 100 numbers, and give a similarity between any two films even if they share no raters.

WHY NOT THIS? (TRADEOFFS)

The factors are not interpretable by design, a new user or film has no factors until it has ratings, and RMSE gains did not become better top-10 lists.



Top: share of the residual table in the top k directions, real against shuffled; validation RMSE of the SVD routes; the ALS grid. Bottom: training (dotted) and validation (solid) RMSE per pass for SGD and ALS; test RMSE of every model; the first 50 singular values, real against shuffled.

Interview preparation — technical questions

Q: Why can't you just run numpy's SVD on a ratings table?

A: The SVD needs every cell. Filling the empty ones with 0 scored 2.4282 on validation; filling with the bias model 0.9365 at rank 20, and fitting only the known ratings did better still.

Q: What does the penalty do in matrix factorization?

A: It keeps factors small unless many ratings support them. Without it, fill-factorize-repeat reached training RMSE 0.2279 and validation 1.0896 at rank 50.

Q: ALS or SGD?

A: Same objective. ALS: exact steps, no learning rate, parallel per user. SGD: streams data, extends to any differentiable model. Here they tied on test (0.9115 and 0.9105).

Q: Why use np.add.at in mini-batch SGD?

A: When an index repeats in a batch, a[idx] += v applies only one update; np.add.at accumulates them all.

Q: How do you show the factors are personal?

A: Shuffle users' factors among users and re-score: 0.9711 here, worse than the bias model's 0.9401.

Interview preparation — theoretical questions

Q: Why is it called 'Funk SVD' when no SVD is computed?

A: Simon Funk's Netflix Prize write-up framed the learned factors as an 'SVD' of the incomplete table; the name stuck.

Q: What is the Eckart-Young theorem?

A: The truncated SVD is the best rank-k approximation in squared error, and its error is the root sum of the discarded squared singular values (219.9119 here, matched by our SVD).

Q: Why did the lowest objective not give the lowest test error?

A: Early stopping limits how far SGD fits the training data, acting as extra regularisation; SGD's objective was higher but its test RMSE slightly lower.

Q: What problem do factors not solve?

A: Cold start: a new user or film has no ratings and hence no factors. Days 156 and 158 address it.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A ratings table is approximately low-rank: a modest share of it is shared structure, and much is noise. Fitting user and film factors to the known ratings only, with a penalty, gave the most accurate predictions so far, while filling the empty cells and factorizing either ignored the missing data's nature or overfitted it. A shuffled-factor control confirmed the factors are personal, and the factor directions read as taste axes.

- Test RMSE: SVD with bias filling 0.9255, fill-factorize-repeat 0.9269, ALS 0.9115, SGD 0.9105, item-based 0.9153.
- Zero filling scored 2.4282 on validation: what goes in the empty cells matters more than the rank.
- No penalty overfits: training 0.2279, validation 1.0896 at rank 50.
- Control with shuffled user factors: 0.9711.

Suggested watch order, if starting from zero

- 1. How does Netflix recommend movies? Matrix Factorization (Luis Serrano Academy) — matrix factorization explained with a small worked table.
- 2. Matrix Completion and the Netflix Prize (Steve Brunton) — matrix completion and the Netflix Prize, from the SVD's point of view.

Day 156 preview

- **Content-based filtering.** Neighbours and factors need ratings; a film nobody has rated has none. Content-based filtering recommends from what the films are (their genres and release year), building a profile of each user's tastes, and so can place a brand-new film.